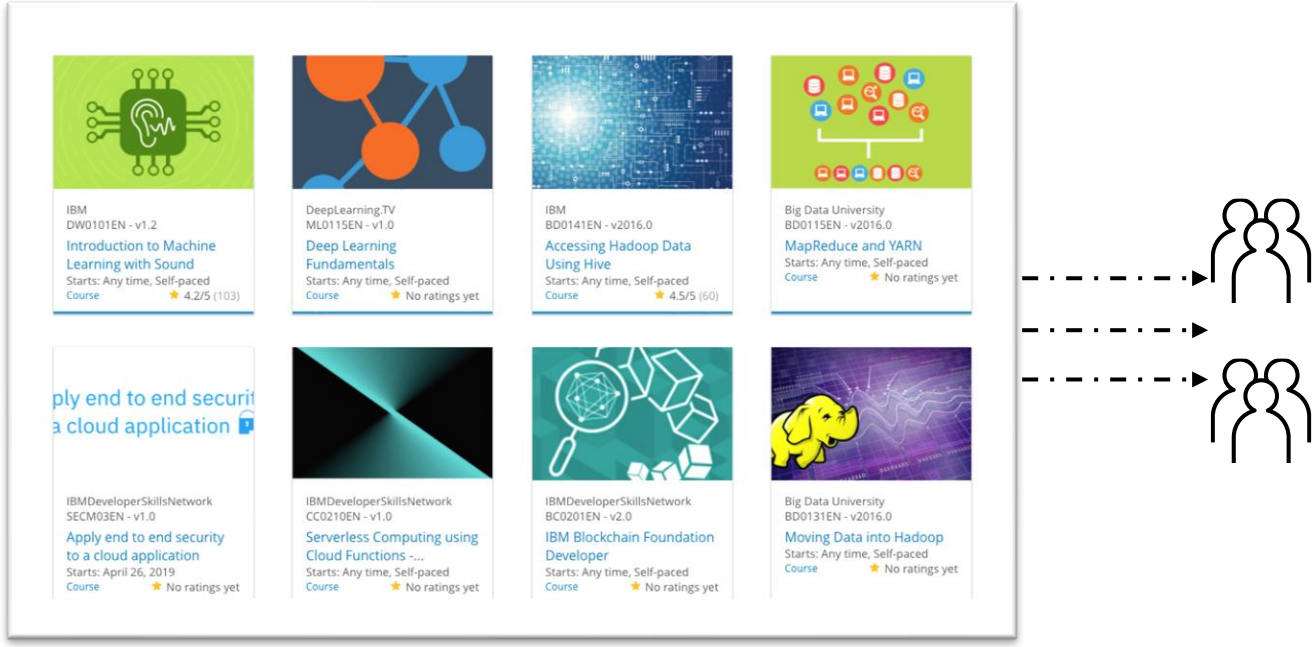
Build a Personalized Online Course Recommender System with Machine Learning

<Sadegh Khajepour>
<2025/03/01>



Outline

- Introduction and Background
- Exploratory Data Analysis
- Content-based Recommender System using Unsupervised Learning
- Collaborative-filtering based Recommender System using Supervised learning
- Conclusion
- Appendix

Introduction

- **Project Background and Context:**

  In this project, we will be developing a content-based recommender system. A content-based recommender uses item features to recommend items similar to those the user has shown interest in. This type of system analyzes the attributes of items and suggests similar items based on the content they contain.

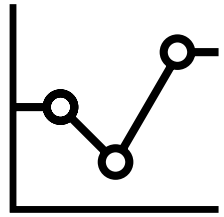
- **Problem Statement:**

  The goal is to build a content-based recommender system that effectively suggests relevant items to users based on their preferences. The challenge lies in selecting and implementing the right models to achieve optimal recommendations.

- **Hypotheses:**

  We hypothesize that different machine learning models, when applied to the task of content-based recommendation, will perform differently. Our objective is to test various models and identify the one that provides the most accurate and relevant recommendations for the users.

Exploratory Data Analysis

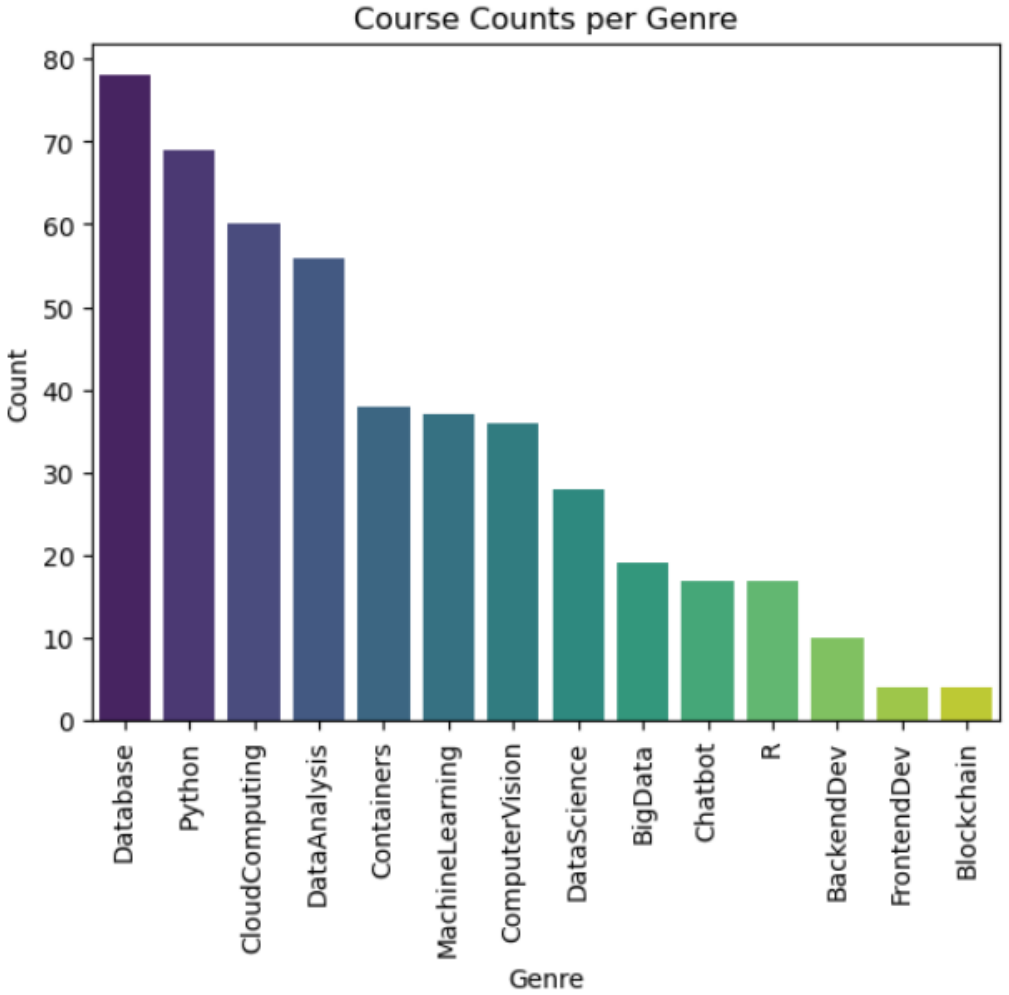


Course counts per genre

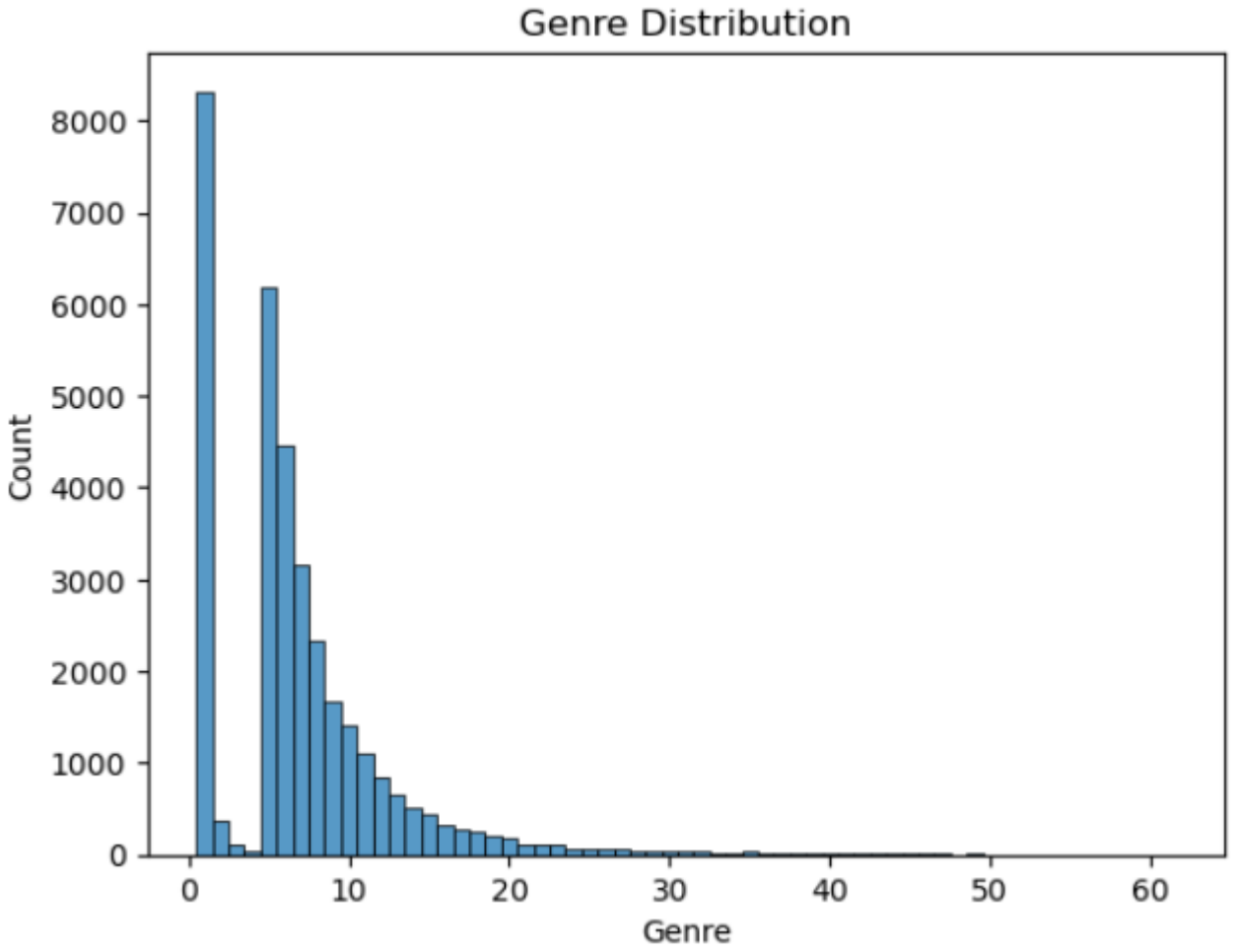
The bar chart shows the number of courses available for different genres in a learning platform.

- **Database** leads with nearly 80 courses, indicating high demand or interest.
- **Python** and **Cloud Computing** follow with around 70 and 60 courses, respectively.
- **Data Analysis** also has a significant presence.
- Mid-tier categories like **Machine Learning**, **Computer Vision**, and **Data Science** each have around 30–40 courses.
- Genres like **Big Data**, **Chatbot**, and **R** have fewer courses, ranging between 10 and 20.
- **Blockchain**, **Frontend Development**, and **Backend Development** have the fewest courses, suggesting less content or lower demand.

The distribution suggests a strong focus on data-related skills, especially databases and Python, while newer or niche fields like Blockchain have less content.



Course enrollment distribution



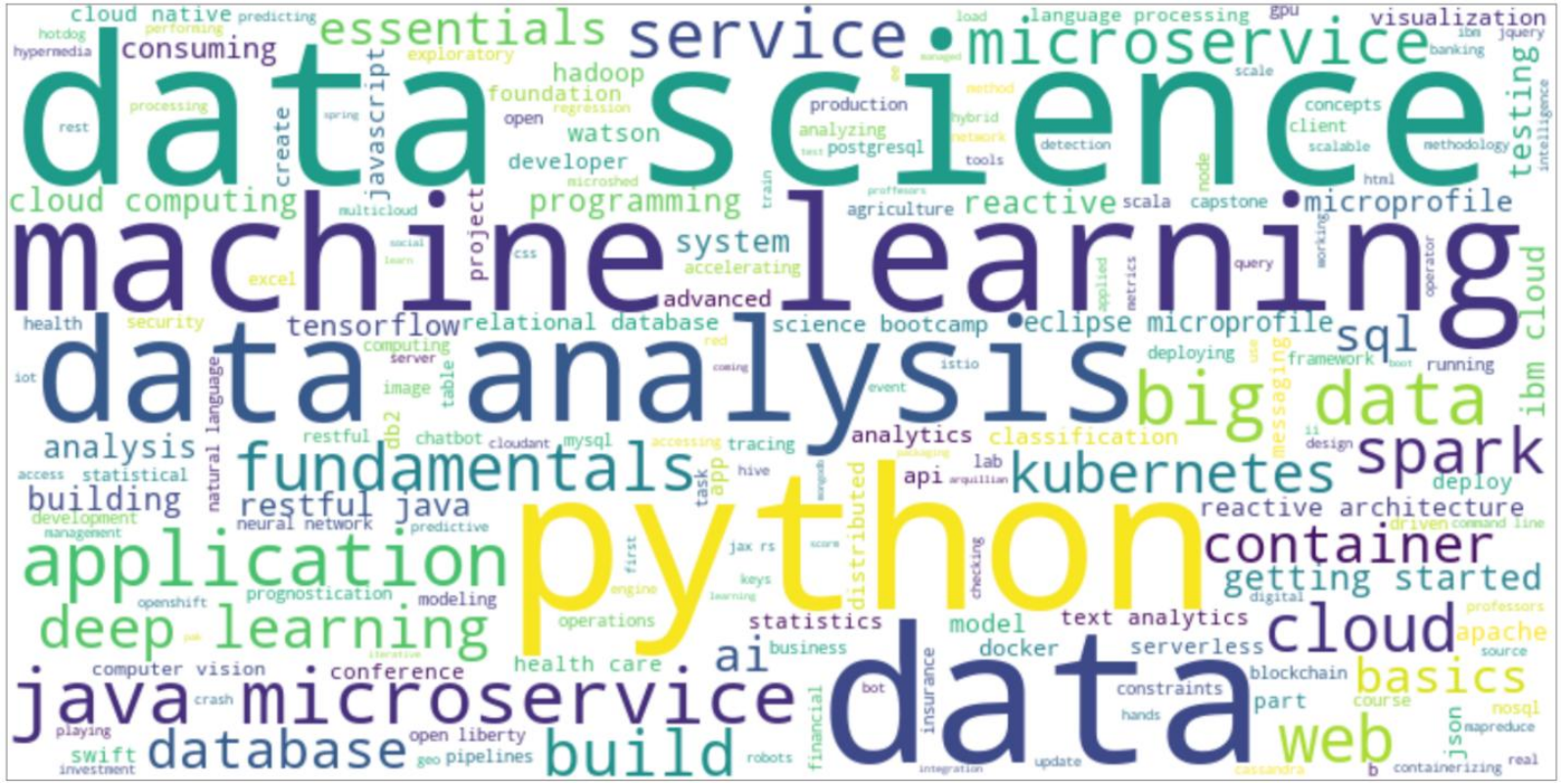
The distribution is highly skewed to the right (positive skew), with a few dominant genres having very high counts (over 8,000 for the most frequent genre) and a long tail of less common genres with decreasing frequencies. The majority of content appears to be concentrated in genres numbered 0-15, with frequencies dropping significantly after that point.

20 most popular courses

The courses primarily focus on data science, programming, and related technologies. Python-based courses dominate the top spots, with "Python for Data Science" having the highest enrollment at 14,936 students. The list demonstrates a strong interest in data science fundamentals, big data technologies, and machine learning, with enrollments gradually decreasing from around 15,000 for the most popular course to about 3,600 for the 20th ranked course.

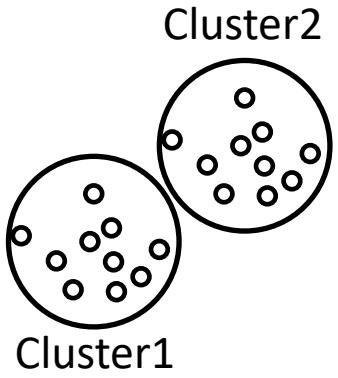
	TITLE	Enrolls
0	python for data science	14936
1	introduction to data science	14477
2	big data 101	13291
3	hadoop 101	10599
4	data analysis with python	8303
5	data science methodology	7719
6	machine learning with python	7644
7	spark fundamentals i	7551
8	data science hands on with open source tools	7199
9	blockchain essentials	6719
10	data visualization with python	6709
11	deep learning 101	6323
12	build your own chatbot	5512
13	r for data science	5237
14	statistics 101	5015
15	introduction to cloud	4983
16	docker essentials a developer introduction	4480
17	sql and relational databases 101	3697
18	mapreduce and yarn	3670
19	data privacy fundamentals	3624

Word cloud of course titles



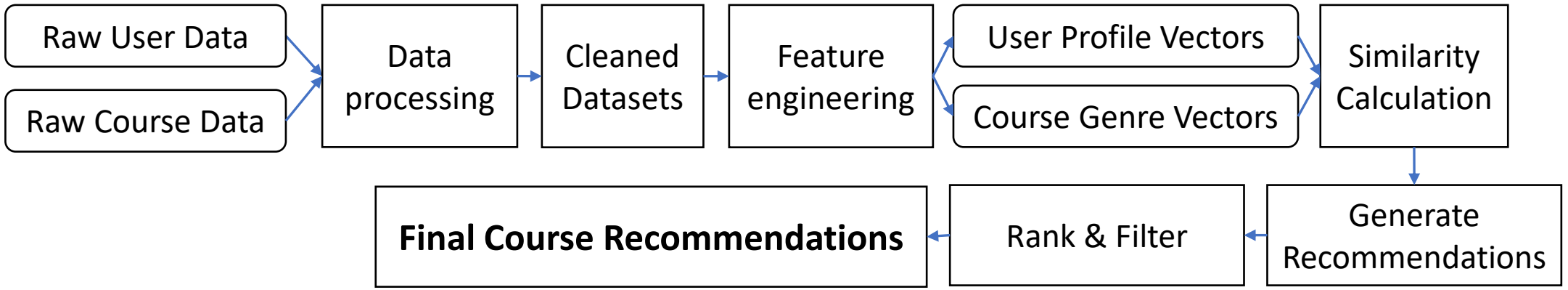
- The largest terms are "**python**", "**data**", "**machine learning**", and "**analysis**", indicating these are the most frequently occurring terms in course titles
- Second-tier prominent terms include "**service**", "**science**", "**java**", "**microservice**", showing important technical and programming concepts
- Supporting terms like "**fundamentals**", "**cloud**", "**big data**", "**kubernetes**", "**spark**" reflect the broader ecosystem of data science and cloud technologies

Content-based Recommender System using Unsupervised Learning



Flowchart of content-based recommender system using user profile and course genres

- **Raw Data Collection:** Starts with raw user data and raw course data.
- **Data Processing:** Both datasets are processed to clean and prepare them for analysis.
- **Feature Engineering:** Cleaned data is transformed into meaningful features, creating user profile vectors and course genre vectors.
- **Similarity Calculation:** The system calculates the similarity between user profiles and course genres.
- **Recommendation Generation:** Based on similarity scores, potential course recommendations are generated.
- **Ranking & Filtering:** Recommendations are ranked and filtered to ensure relevance.
- **Final Output:** The process concludes with a list of final course recommendations tailored to the user.



Evaluation results of user profile-based recommender system

A score threshold of 10.0 is applied to filter recommendations, ensuring that only courses with a recommendation score of 10.0 or higher are included in the final results. This threshold can be adjusted to fine-tune the number and quality of the generated recommendations.

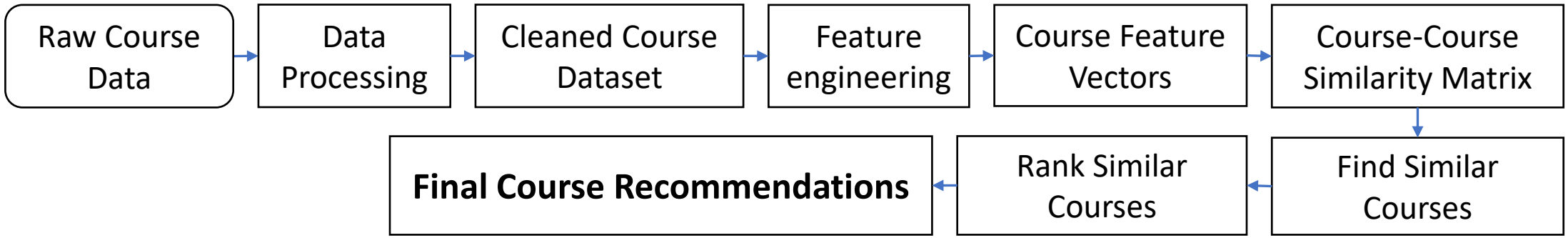
On average, each user receives 60.82 new or unseen course recommendations.

Top 10 most frequently recommended courses:

Title	Count
introduction to data science in python	10343
applied machine learning in python	9291
accelerating deep learning with gpu	8246
foundations for big data analysis with sql	7718
analyzing big data in r using apache spark	7373
machine learning with python	7285
getting started with the data apache spark makers build	7066
analyzing big data with sql	7008
spark overview for scala analytics	6355
spark fundamentals ii	5725

Flowchart of content-based recommender system using course similarity

- **Raw Course Data:** Starts with raw course data.
- **Data Processing:** Data is cleaned and prepared.
- **Feature Engineering:** Cleaned data is transformed into course feature vectors.
- **Similarity Matrix:** A course-course similarity matrix is calculated.
- **Find Similar Courses:** Identifies courses with high similarity.
- **Rank Courses:** Ranks similar courses by relevance.
- **Final Recommendations:** Produces a list of recommended courses.



Evaluation results of course similarity based recommender system

Use a threshold of 0.6 for filtering recommendations. This ensures that only courses with a similarity score of 0.6 or higher are recommended, balancing relevance and the number of recommendations.

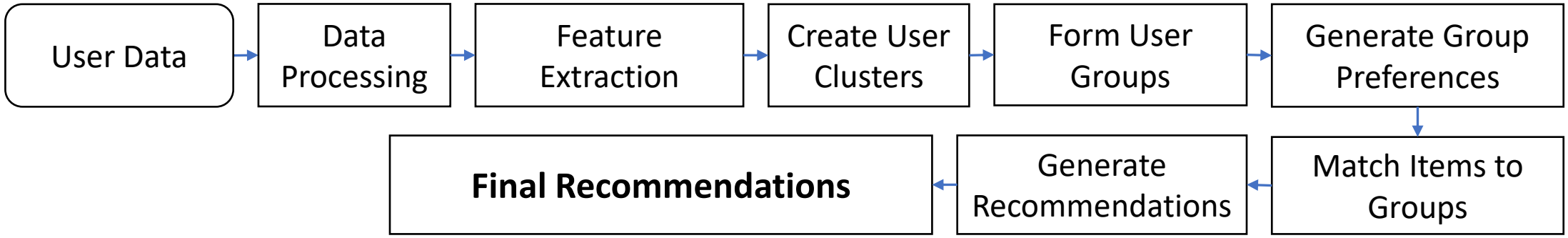
Each user receives, on average, 9.77 new or previously unseen course recommendations.

Top 10 most frequently recommended courses:

Title	Count
introduction to data science in python	29874
data analysis using python	17052
data science with open data	15003
data science fundamentals for data analysts	14641
a crash course in data science	14641
big data modeling and management systems	13551
foundations for big data analysis with sql	13512
introduction to big data	13291
fundamentals of big data	13291
sql access for hadoop	12497

Flowchart of clustering-based recommender system

- **User Data:** Begins with raw user data.
- **Data Preprocessing:** The data is cleaned and prepared for analysis.
- **Feature Extraction:** Cleaned data is transformed into user feature vectors.
- **Create User Clusters:** Users are grouped into clusters based on similarities.
- **Form User Groups:** Clusters are organized into distinct user groups.
- **Generate Group Preferences:** Preferences are derived for each user group.
- **Match Items to Groups:** Items are matched to the preferences of each group.
- **Generate Recommendations:** Recommendations are generated for each group.
- **Final Recommendations:** A final list of recommendations is produced for users.



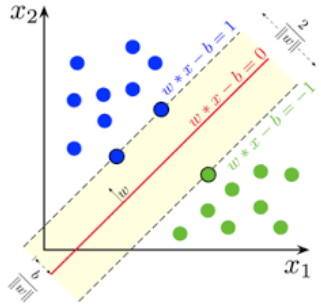
Evaluation results of clustering-based recommender system

Selected 3 items with the highest enrollments for each cluster

The analysis of course enrollments across different clusters reveals distinct patterns in learner preferences. For each cluster, the top three courses with the highest enrollments were identified.

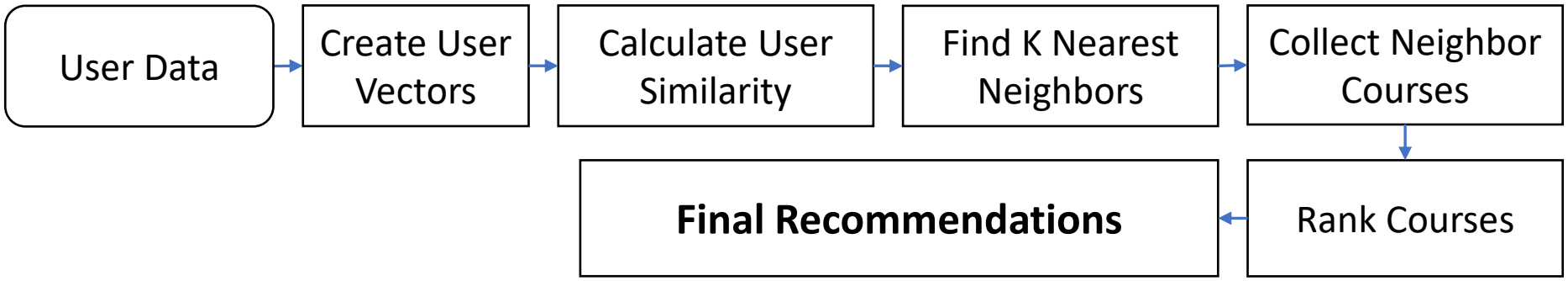
Cluster	Course 1	Course 2	Course 3
Cluster 0	CB0103EN	PY0101EN	DS0101EN
Cluster 1	PY0101EN	DS0101EN	DA0101EN
Cluster 2	LB0101ENv1	LB0103ENv1	LB0105ENv1
Cluster 3	BD0101EN	BD0111EN	PY0101EN
Cluster 4	GPXX0T0FEN	PY0101EN	ML0101ENv3
Cluster 5	DS0101EN	BD0101EN	CNSC02EN
Cluster 6	CO0101EN	CO0201EN	CC0101EN
Cluster 7	BD0111EN	BD0101EN	BD0211EN

Collaborative-filtering Recommender System using Supervised Learning



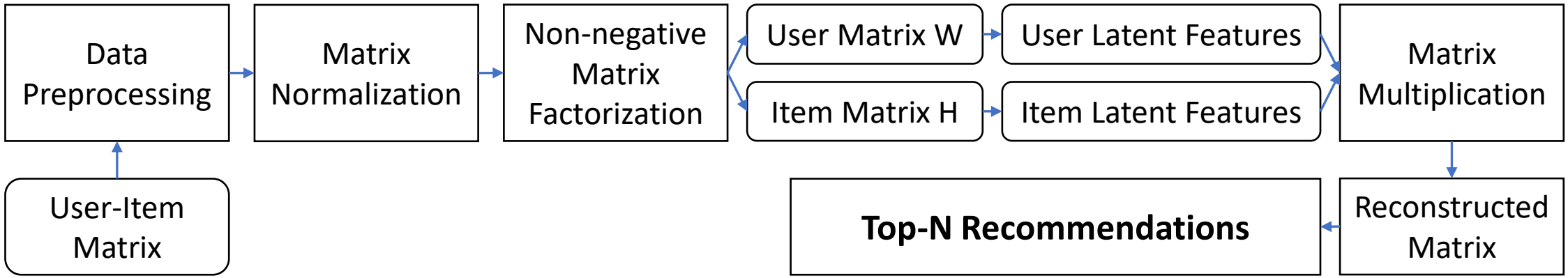
Flowchart of KNN based recommender system

- **User Enrollment Data:** Begins with raw user enrollment data.
- **Create User Vectors:** User data is transformed into feature vectors for analysis.
- **Calculate User Similarity:** Similarity between users is computed based on their feature vectors.
- **Find K Nearest Neighbors:** Identifies the most similar users (neighbors) for each user.
- **Collect Neighbor Courses:** Gathers courses enrolled in by the nearest neighbors.
- **Rank Courses:** Ranks the collected courses based on relevance and popularity.
- **Final Recommendations:** Produces a final list of personalized course recommendations for users.



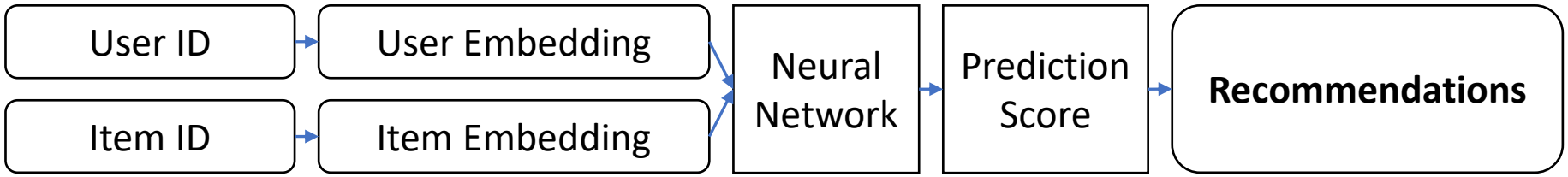
Flowchart of NMF based recommender system

- **User-Item Matrix:** Begins with the raw user-item interaction data.
- **Data Preprocessing:** The data is cleaned and prepared for analysis.
- **Matrix Normalization:** Normalizes the user-item matrix to ensure consistent scaling.
- **Non-negative Matrix Factorization (NMF):** Decomposes the normalized matrix into two latent factor matrices.
  - **User Matrix W:** Represents user latent features.
  - **Item Matrix H:** Represents item latent features.
- **User Latent Features:** Extracts latent features for users from matrix W.
- **Item Latent Features:** Extracts latent features for items from matrix H.
- **Matrix Multiplication:** Combines user and item latent features to reconstruct the user-item matrix.
- **Reconstructed Matrix:** Generates a refined user-item interaction matrix with predicted preferences.
- **Top-N Recommendations:** Produces the final list of top-N personalized recommendations for users.



Flowchart of Neural Network Embedding based recommender system

- **User ID:** Begins with the unique identifier for each user.
- **User Embedding:** Transforms the user ID into a dense vector representation (embedding) for analysis.
- **Item ID:** Begins with the unique identifier for each item.
- **Item Embedding:** Transforms the item ID into a dense vector representation (embedding) for analysis.
- **Neural Network:** Combines user and item embeddings as input to a neural network for learning interactions.
- **Prediction Score:** Generates a score representing the likelihood of user-item interaction.
- **Recommendations:** Produces a final list of personalized recommendations based on the prediction scores.

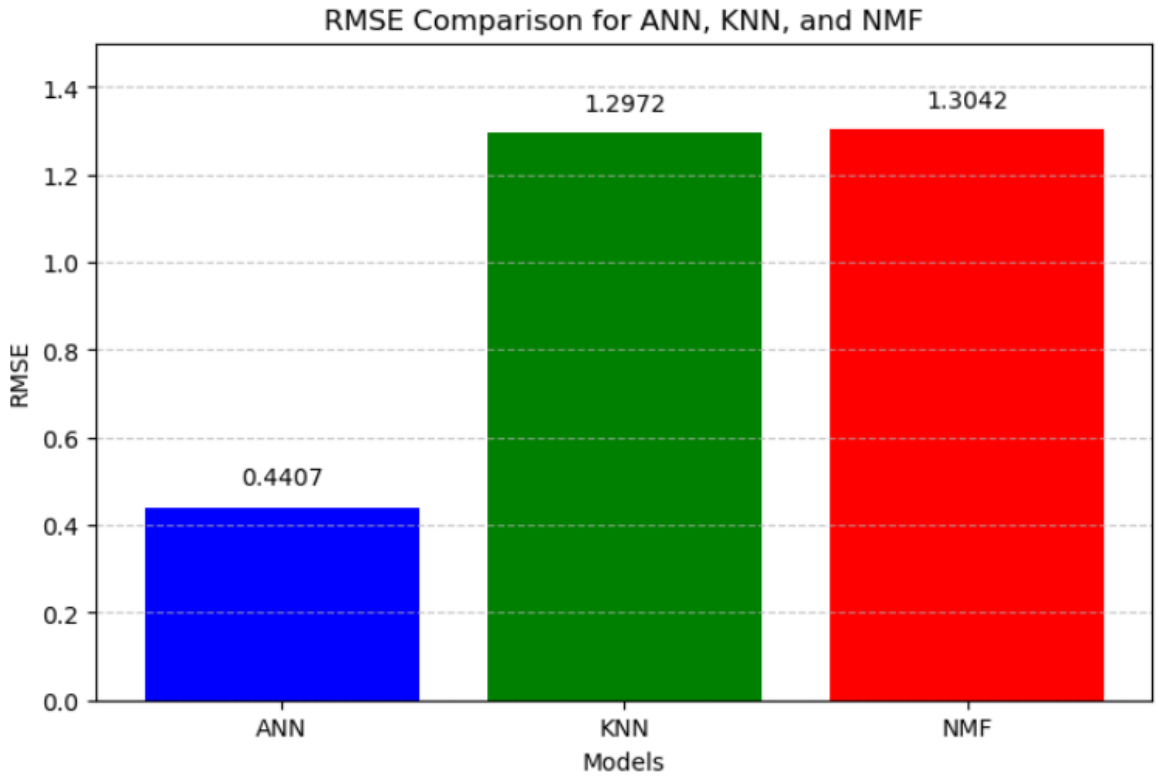


Compare the performance of collaborative-filtering models

The bar chart compares the RMSE values for three recommendation models: ANN, KNN, and NMF.

ANN achieves the lowest RMSE, demonstrating superior accuracy in predicting user preferences compared to KNN and NMF, which show similar performance.

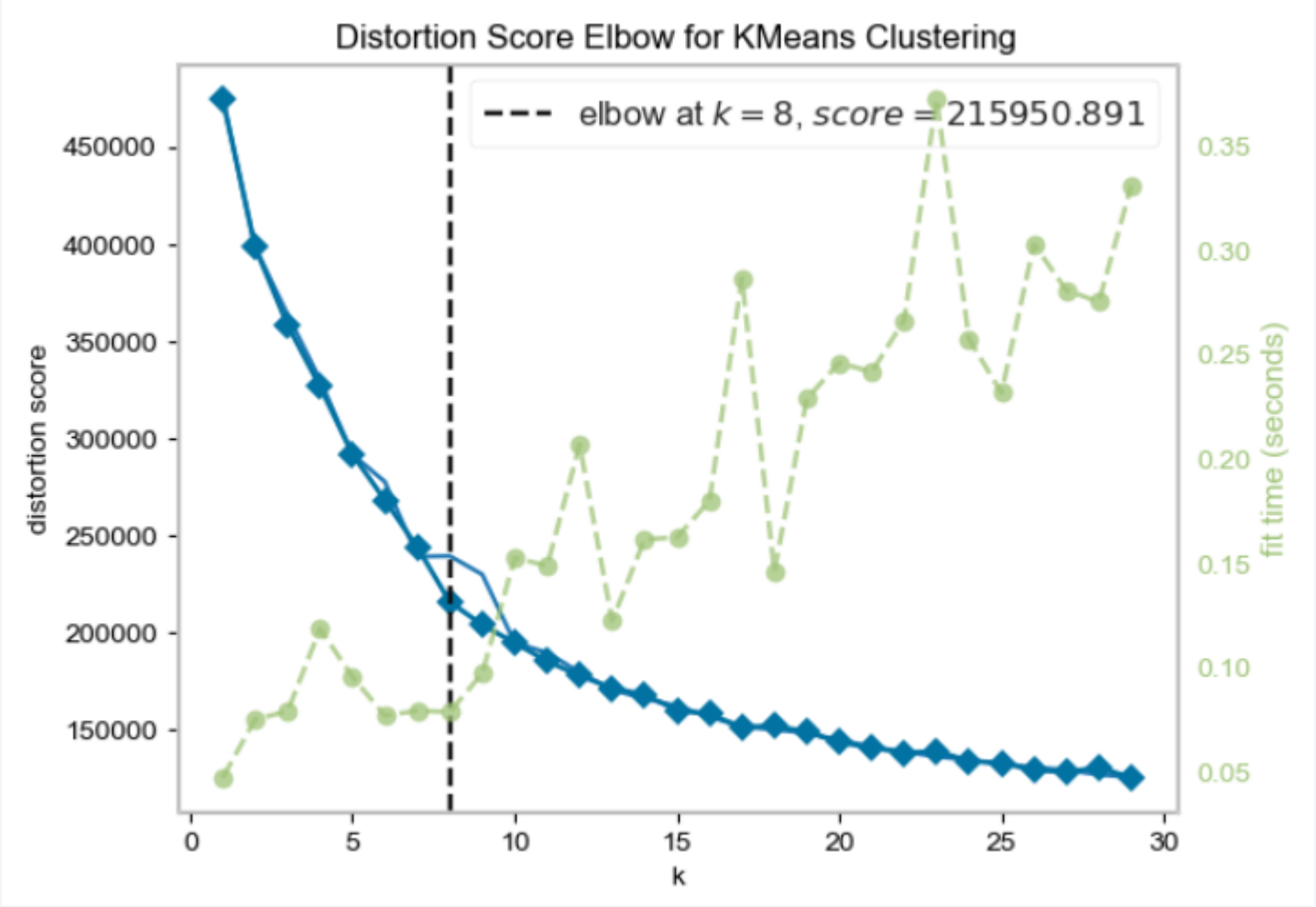
This highlights ANN's effectiveness in generating more precise recommendations over traditional methods.



Conclusions

- Exploratory data analysis was conducted on the user-item interaction datasets.
- Content-based recommendation systems were implemented using unsupervised models.
- Supervised learning methods were ultimately employed to develop collaborative filtering-based recommender systems.
- For the content-based systems, adjusting and optimizing the similarity threshold was achieved by evaluating the average number of recommendations per user. An average of approximately 20 recommendations per user was found to be optimal.
- Among the collaborative filtering-based systems, ANN (Artificial Neural Networks) demonstrated the best performance.

Appendix


Distortion Score Elbow for KMeans Clustering
elbow at $k = 8$, $score = 215950.891$
distortion score
fit time (seconds)
k
450000
400000
350000
300000
250000
200000
150000
0.35
0.30
0.25
0.20
0.15
0.10
0.05
0
5
10
15
20
25
30